FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

COMPUTER SCIENCE DEPARTMENT

Project Specification
CS0035 - PROGRAMMING LANGUAGES

RESEARCH PROJECTS

A compiler translates (or compiles) a program written in a high-level programming language that is suitable for human programmers into the low-level machine language that is required by computers.

Compiler design covers basic translation mechanisms and error detection & recovery. It includes lexical, syntax, and semantic analysis as front end, and code generation and optimization as back-end.

A **lexical analysis** stage that basically divides the input text into a list of "words". This is the initial part of reading and analyzing the program text: The text is read and divided into tokens, each of which corresponds to a symbol in the programming language, e.g., a variable name, keyword or number.

The next process will be a **syntax analysis** stage that analyses the way these words form structures and converts the text into a data structure that reflects the textual structure. For finding the structure of the input text, the syntax analysis must also reject invalid texts by reporting syntax errors. This phase takes the list of tokens produced by the lexical analysis and arranges these in a tree- structure (called the syntax tree) that reflects the structure of the program. This phase is often called parsing.

The following set of rules that define for the syntax:

1) All statements will end in a **semicolon**.
2) **Whitespace is not significant**, i.e. *a+b* is the same as *a + b*.
3) Comments begin with ***/*** and end with ****/*** and they can extend across multiple lines.
4) The only keywords are *var*, *input*, and *output*. Their function is as follows:

   var declares a new variable. All variables must be declared before they're used.

   input reads a variable from the console.

   output displays the value of a variable on the console.

5) All variables are integers.
6) The only operations allowed are +, -, *, /, and =. Parentheses may be used to force operation ordering.
7) Variable names may only consist of uppercase and lowercase letters, underscores (_), and numbers. A variable name may *not* start with a number.

After the syntax analysis being performed the **type checking,** this phase analyses the syntax tree to determine if the program violates certain consistency requirements, e.g., if a variable is used but not declared or if it is used in a context that does not make sense given the type of the variable, such as trying to use a boolean value as a function pointer.

Design: invent a language, or a language fragment, for some particular purpose or with some particular characteristic.

Modeling: try to formalize some interesting aspect of some existing language. Make sure that computations are sufficiently documented and supported by theories and literature.

Implementation: explore novel techniques for implementing a given language fragment

D. May choose any programming language. (C, C++, Java, C#, Python, etc)
E. A copy of the software (executable and codes) needs to be submitted.

FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Project JACK: The Official Billing System of JanTayoBibili Shop

A project submitted to
Dr. Angelo C. Arguson
Computer Science Department
FEU Institute of Technology

In partial fulfillment of the requirements in
CS0035 - PROGRAMMING LANGUAGES
for 2nd Trimester, Academic Year 2025-2026

OPULENCIA, KAISER G.

FACTO, JANELLE P.

MAGBAG, COLEEN RAYE C.

UY, AISLINN H.

TN37

MARCH 20, 2026

I. Project Overview

Project JACK is a simple custom Domain-Specific Language (DSL) made for the billing system of "JanTayoBibili Shop." It works like a basic compiler and interpreter written in Python, where it reads a text file (.txt) as source code, checks for errors, and then executes it. The system can compute the subtotal of items like laptops, tablets, and phones, apply a fixed discount, and calculate monthly installment payments. It also includes error handling, such as detecting missing semicolons or undeclared variables, as shown in the incorrect input file.

II. Technical Specifications

The Jack language is built upon a four-phase architectural pipeline:

A. Lexical Analysis

This phase tokenizes the raw input string into categorized components while filtering out whitespaces and comments.

- Keywords: var, input, output.
- Operators: =, +, -, *, /.
- Delimiters: ;, (,).
- Identifiers: Alphanumeric names (cannot start with a number).

B. Syntax Analysis

The phase receives tokens and ensures they follow the defined Jack grammar. It constructs an Abstract Syntax Tree (AST) using recursive-descent parsing.

- Rules:
  - Every statement must end with a semicolon (;).
  - Parentheses must be balanced.
  - Expressions follow correct operator order.

C. Semantic Analysis

This phase checks if the program is logically correct before execution.

- Declaration Check: Ensures every variable is declared with var before being used in assignments or outputs.
- Duplicate Check: Prevents redeclaring the same variable name.
- Existence Check: All variables used in input, output, and expressions must exist.

D. Code Interpretation

This executes the program by traversing the AST. It stores variable values in a simple environment like a dictionary and performs calculations based on standard operator precedence. The interpreter is capable of the following:

- Supports integer operations: addition, subtraction, multiplication, and division.
- Accepts user input and displays computed results.
- Handles runtime errors such as invalid input or division by zero.

III. Code Analysis and Functions

- **Code Analysis:**
  - **jack_language.txt file**
    - **Description:** This demonstrates how the JACK language expresses logic for billing operations. This begins by declaring all necessary variables, followed by input statements that collect quantities of laptops, tablets, and phones from the user. It then performs a series of calculations, including subtotal computation using multiplication and addition, application of a fixed discount, and division to determine monthly installment payments. The file also includes structured comments that are ignored during lexical analysis.
    - **Comments Block**
      - All codes here are inside comment blocks. The lexer skips these entirely using skip_comment() and does not affect executions.
    - **Variable Declarations**

```
/* Variable Declarations */
var laptop_count
var tablet_count;
var phone_count;
var sub_total;
var discount_amount;
var net_total;
var monthly_payment;
```

      - Declares all variables beforehand and are checked in semantic analysis.

- **Input Statements**

```
/* Input Quantity from User */
input laptop_count;
input tablet_count;
input phone_count;
```

  - Prompts the user to enter values and store values in the environment dictionary in the interpreter phase.

- **Subtotal Computation**

```
/* Sub-total Calculation (Multiplication and Addition) */
/* Prices: Laptop = 26000, Tablet = 10000, Phone = 7000 */
sub_total = (laptop_count * 26000) + (tablet_count * 10000) + (phone_count * 7000);
```

  - Uses multiplication, addition, and parenthesis for grouping, which are parsed into a BinOpNode tree and evaluated using recursive expression evaluation. This calculates the sub-total cost of the customer's purchase.

- **Discount Logic**

```
/* Apply Fixed Discount (Subtraction) */
discount_amount = 1500;
net_total = sub_total - discount_amount;
```

  - Uses simple assignment and subtraction and shows reuse of variables. This applies a discount to the customer's purchase.

- **Installment Calculation**

```
/* Calculate 12-Month Installment (Division) */
monthly_payment = net_total / 12;
```

  - Uses integer division. This calculates the payment amount for the user to pay the net total in 12 equal monthly installments.

- **Output Statements**

```
/* Display results */
output sub_total;
output discount_amount;
```

```
output net_total;
output monthly_payment;
```

- This displays computed results.

- **jack_wrong.txt file**
  - **Description:** This is an intentionally incorrect version of the Jack source code used to demonstrate error detection capabilities of the system. It has a syntax error caused by a missing semicolon in one of the variable declarations, which breaks the grammar rules of the language. This file is used to test the Syntax Analysis phase of the system, where the parser checks for errors in the structure of the code. By using an incorrect input like this, the system shows that it can detect invalid programs and give clear error messages instead of running them, which helps improve reliability and user understanding.
  - **Incorrect Part:** var laptop_count
- **JACK.py file**
  - **Description:** This integrates all four phases of the compiler-interpreter pipeline: lexical analysis, syntax analysis, semantic analysis, and code interpretation. It begins with the Lexer, which tokenizes the input source code by identifying keywords, operators, delimiters, integers, and identifiers while ignoring whitespace and comments. The Parser then processes these tokens using a recursive-descent approach to construct an Abstract Syntax Tree (AST), ensuring that the program follows the defined grammar rules. The Semantic Analyzer validates the logical correctness of the program by checking for proper variable declarations and preventing duplicate definitions. The Interpreter executes the AST through a tree-walk mechanism, storing variable values in a dictionary-like environment, performing arithmetic operations, handling user input and output, and detecting runtime errors such as invalid input or division by zero. Additionally, the file includes a driver function that starts the

entire compilation and execution process, providing a structured and sequential flow from source code input to final output.

- **Program Overview**
  - **Comment Block**

```
"""
CSTPLANGS - Simple Language Compiler/Interpreter
FEU Institute of Technology - Computer Science Department

Phases:
  1. Lexer - Tokenizes input source code
  2. Parser - Builds an Abstract Syntax Tree (AST)
  3. Semantic - Type checks and validates declarations
  4. Interpreter - Executes the AST
"""
```

    - This comment block outlines its four main phases: Lexer, Parser, Semantic Analysis, and Interpreter.
  - **System and Module Imports**

```
import sys
import emoji
```

    - This imports the necessary modules, including sys for handling command-line arguments and emoji for display functionality.
- **Phase 1: Lexer (Tokenizer)**
  - **Comment Block**

```
# ==================================================================
# PHASE 1: LEXER (Tokenizer)
# ==================================================================
```

    - Comments the phase

- **Token Types**

```
# Token types
TT_VAR    = 'VAR'
TT_INPUT  = 'INPUT'
TT_OUTPUT = 'OUTPUT'
TT_IDENT  = 'IDENTIFIER'
TT_INT    = 'INTEGER'
TT_PLUS   = 'PLUS'
TT_MINUS  = 'MINUS'
TT_MUL    = 'MUL'
TT_DIV    = 'DIV'
TT_ASSIGN = 'ASSIGN'
TT_SEMI   = 'SEMICOLON'
TT_LPAREN = 'LPAREN'
TT_RPAREN = 'RPAREN'
TT_EOF    = 'EOF'

KEYWORDS = {'var', 'input', 'output'}
```

  - This block defines all possible token types in the language and the reserved keywords that the lexer must recognize. These tokens represent the basic elements that will later be used by the parser.

- **Token Class**

```
class Token:
    def __init__(self, type_, value, line):
        self.type  = type_
        self.value = value
        self.line  = line

    def __repr__(self):
                return  f'Token({self.type},
{self.value!r}, line={self.line})'
```

  - The Token class represents a single token in the source code. It stores the token's type, value, and the line number where it appears, which is useful for debugging and error reporting.

- **Exception Classes**

```
class LexerError(Exception):
    pass
```

```
class ParseError(Exception):
    pass

class SemanticError(Exception):
    pass

class RuntimeError_(Exception):
    Pass
```

- These custom exception classes are used to handle different types of errors in each phase of the compiler: lexical, syntax, semantic, and runtime errors.

- **Lexer Class**

```
class Lexer:
    def __init__(self, source):
        self.source = source
        self.pos    = 0
        self.line   = 1
        self.tokens = []
```

- Initialization
  - This initializes the lexer with the source code, starting position, current line number, and an empty list where generated tokens will be stored.

```
 def error(self, msg):
             raise LexerError(f"[Lexer] Line
{self.line}: {msg}")
```

- Error Handling
  - This function is used by the lexer to handle errors during tokenization. When an invalid character or issue is encountered, it raises a LexerError and includes the current line number in the message. This helps in identifying exactly where the error occurred in the source code, making debugging easier.

```
def peek(self):
```

```
            return self.source[self.pos] if self.pos < len(self.source) else None

    def advance(self):
        ch = self.source[self.pos]
        self.pos += 1
        if ch == '\n':
            self.line += 1
        return ch

    def skip_whitespace(self):
        while self.peek() and self.peek() in ' \t\r\n':
            self.advance()
```

- Helper Functions
  - peek() looks at the current character without moving forward.
  - advance() moves the pointer forward and updates the line number.
  - skip_whitespace() ignores spaces, tabs, and newlines.

```
def skip_comment(self):
        """Skip /* ... */ block comments (may span multiple lines)."""
        self.advance()  # consume '/'
        self.advance()  # consume '*'
        while self.pos < len(self.source):
            if self.source[self.pos] == '*' and self.pos + 1 < len(self.source) and self.source[self.pos + 1] == '/':
                self.advance()  # consume '*'
                self.advance()  # consume '/'
                return
            self.advance()
            self.error("Unterminated block comment")
```

- Skipping Comments
  - This function skips block comments enclosed in /* ... */. If the closing */ is missing, it raises an error.

```
    def read_number(self):
        start = self.pos
                    while   self.peek()   and
self.peek().isdigit():
            self.advance()
        return self.source[start:self.pos]
```

- Reading Numbers
  - This reads a sequence of digits and converts it into an integer token.

```
    def read_identifier(self):
        start = self.pos
                        while   self.peek()   and
(self.peek().isalnum() or self.peek() == '_'):
            self.advance()
        return self.source[start:self.pos]
```

- Reading Identifiers
  - This reads a sequence of digits and converts it into an integer token.

```
def tokenize(self):
        simple = {
            '+': TT_PLUS,
            '-': TT_MINUS,
            '*': TT_MUL,
            '/': TT_DIV,
            '=': TT_ASSIGN,
            ';': TT_SEMI,
            '(': TT_LPAREN,
            ')': TT_RPAREN,
        }

        while self.pos < len(self.source):
            self.skip_whitespace()
            if self.pos >= len(self.source):
                break

            ch = self.peek()

            # Block comment
                if ch == '/' and self.pos + 1 <
len(self.source) and self.source[self.pos + 1] ==
'*':
                self.skip_comment()
```

```
                continue

            # Integer literal
            if ch.isdigit():
                line = self.line
                num  = self.read_number()
                    self.tokens.append(Token(TT_INT,
int(num), line))
                continue

            # Identifier or keyword
            if ch.isalpha() or ch == '_':
                line  = self.line
                ident = self.read_identifier()
                if ident in KEYWORDS:
                        tt = {'var': TT_VAR, 'input':
TT_INPUT, 'output': TT_OUTPUT}[ident]
                         self.tokens.append(Token(tt,
ident, line))
                else:

self.tokens.append(Token(TT_IDENT, ident, line))
                continue

            # Single-character tokens
            if ch in simple:
                line = self.line
                self.advance()

self.tokens.append(Token(simple[ch], ch, line))
                continue

                 self.error(f"Unexpected character:
{ch!r}")

             self.tokens.append(Token(TT_EOF,  None,
self.line))
        return self.tokens
```

- Main Tokenization Function
  - This converts the entire source code into tokens. It identifies numbers, identifiers, keywords, operators, and symbols, then

stores them in a list. If an invalid character is found, it raises an error. Finally, it adds an EOF token to mark the end of the input.

- Phase 2: Parser (Recursive-Descent to AST)
  - **Comment Block**

```
#
==========================================================================
# PHASE 2: PARSER  (Recursive-Descent → AST)
#
==========================================================================
```

    - Comments the phase
  - AST node classes
    - The parser defines several classes that represent different elements of the Abstract Syntax Tree (AST). Each node corresponds to a specific construct in the language:

```
class VarDeclNode:
    """var <name> ;"""
    def __init__(self, name, line):
        self.name = name
        self.line = line
```

    - VarDeclNode
      - Represents a variable declaration and stores the variable name and its line number.

```
class AssignNode:
    """<name> = <expr> ;"""
    def __init__(self, name, expr, line):
        self.name = name
        self.expr = expr
        self.line = line
```

    - AssignNode
      - Represents assignment statements and stores the target variable and its

associated expression.

```
class InputNode:
    """input <name> ;"""
    def __init__(self, name, line):
        self.name = name
        self.line = line
```

- InputNode
  - Represents user input operations.

```
class OutputNode:
    """output <name> ;"""
    def __init__(self, name, line):
        self.name = name
        self.line = line
```

- OutputNode
  - Represents output statements.

```
class BinOpNode:
    """<left> op <right>"""
    def __init__(self, left, op, right):
        self.left  = left
        self.op    = op
        self.right = right
```

- BinOpNode
  - Represents binary operations such as addition, subtraction, multiplication, and division.

```
class NumberNode:
    def __init__(self, value):
        self.value = value
```

- NumberNode
  - Represents numeric values in expressions.

```
class IdentNode:
    def __init__(self, name, line):
        self.name = name
        self.line = line
```

- IdentNode
  - Represents variables used in expressions, along with their line numbers.

- Parser Class

```
class Parser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.pos    = 0
```

  - Initialization
    - The parser receives a list of tokens from the lexer and initializes a pointer (pos) to track the current token being processed.

- Helper Functions

```
    def current(self):
        return self.tokens[self.pos]
```

  - current() returns the current token.

```
    def eat(self, type_):
        tok = self.current()
        if tok.type != type_:
                self.error(f"Expected {type_},
got {tok.type} ({tok.value!r})")
        self.pos += 1
        return tok
```

  - eat() checks if the current token matches the expected type. If it does, it moves to the next token; otherwise, it raises a syntax error.

```
    def peek_type(self):
        return self.current().type
```

  - peek_type() returns the type of the current token without advancing.

- Error Handling

```
    def error(self, msg):
        tok = self.current()
            raise ParseError(f"[Parser] Line
{tok.line}: {msg}")
```

  - This function raises a syntax error with the corresponding line number when the parser encounters invalid grammar.

- Main Parsing Function

```
    def parse(self):
        """program → statement* EOF"""
```

```
        stmts = []
        while self.peek_type() != TT_EOF:
            stmts.append(self.statement())
        return stmts
```

- This function serves as the entry point of the parser. It processes all tokens and builds a list of statements until it reaches the end of the file (EOF).

- Statement Parsing

```
    def statement(self):
        tt = self.peek_type()
        if tt == TT_VAR:
            return self.var_decl()
        if tt == TT_INPUT:
            return self.input_stmt()
        if tt == TT_OUTPUT:
            return self.output_stmt()
        if tt == TT_IDENT:
            return self.assign_stmt()
               self.error(f"Unexpected  token:
{self.current().value!r}")
```

- This function determines which type of statement to parse based on the current token. If the token does not match any valid statement type, it raises a parsing error using self.error(). This ensures that only valid syntax is processed and helps catch unexpected or invalid tokens early in the parsing phase.
- Variable Declaration

```
    def var_decl(self):
        """var <ident> ;"""
        line = self.current().line
        self.eat(TT_VAR)
        name_tok = self.eat(TT_IDENT)
        self.eat(TT_SEMI)
                                    return
VarDeclNode(name_tok.value, line)
```

  - This function handles variable

declarations in the language. It ensures that the statement follows the correct syntax by requiring the var keyword, followed by a valid identifier and a terminating semicolon. Once validated, it creates a VarDeclNode to represent the declared variable in the AST.

- Input Statement

```
    def input_stmt(self):
        """input <ident> ;"""
        line = self.current().line
        self.eat(TT_INPUT)
        name_tok = self.eat(TT_IDENT)
        self.eat(TT_SEMI)
                                    return
InputNode(name_tok.value, line)
```

  - This function processes input operations where the program accepts user-provided values. It validates the presence of the input keyword, followed by a valid identifier and a semicolon. It then generates an InputNode, which represents the operation of assigning a value from user input to a variable.

- Output Statement

```
    def output_stmt(self):
        """output <ident> ;"""
        line = self.current().line
        self.eat(TT_OUTPUT)
        name_tok = self.eat(TT_IDENT)
        self.eat(TT_SEMI)
                                    return
OutputNode(name_tok.value, line)
```

  - This function handles output operations that display values to the user. It ensures that the statement begins with the output keyword, followed by a valid

identifier and a semicolon. After validation, it creates an OutputNode that represents the instruction to display the value of a variable.

- Assignment Statement

```
    def assign_stmt(self):
        """<ident> = <expr> ;"""
        line = self.current().line
        name_tok = self.eat(TT_IDENT)
        self.eat(TT_ASSIGN)
        expr = self.expr()
        self.eat(TT_SEMI)
                                    return
AssignNode(name_tok.value, expr, line)
```

  - This function manages assignment operations, where a variable is assigned the result of an expression. It verifies that the statement begins with an identifier, followed by the assignment operator (=), an اظهار expression, and a semicolon. It then constructs an AssignNode that links the variable to its corresponding computed expression in the AST.

- Comment Block

```
        # --- Expression parsing (operator
precedence) ---
    # expr   → term   (('+' | '-') term)*
    # term   → factor (('*' | '/') factor)*
    # factor → NUMBER | IDENT | '(' expr ')'
```

  - This describes the grammar rules for expression parsing in the parser. It defines how mathematical expressions are structured and evaluated using operator precedence.

- Expression Parsing
  - expr()

```
    def expr(self):
```

```
        node = self.term()
            while self.peek_type() in (TT_PLUS, TT_MINUS):
                op = self.eat(self.peek_type())
                rhs = self.term()
                node = BinOpNode(node, op.value, rhs)
        return node
```

- Handles addition and subtraction.

- term()

```
    def term(self):
        node = self.factor()
            while self.peek_type() in (TT_MUL, TT_DIV):
                op = self.eat(self.peek_type())
                rhs = self.factor()
                node = BinOpNode(node, op.value, rhs)
        return node
```

- Handles multiplication and division.

- factor()

```
    def factor(self):
        tok = self.current()
        if tok.type == TT_INT:
            self.eat(TT_INT)
            return NumberNode(tok.value)
        if tok.type == TT_IDENT:
            self.eat(TT_IDENT)
            return IdentNode(tok.value, tok.line)
        if tok.type == TT_LPAREN:
            self.eat(TT_LPAREN)
            node = self.expr()
            self.eat(TT_RPAREN)
            return node
        self.error(f"Expected number, identifier, or '(' — got {tok.value!r}")
```

- Handles numbers, variables, and grouped expressions using parentheses.

- Phase 3: Semantic Analysis
  - Comment Block

```
#
==================================================
==========================

# PHASE 3: SEMANTIC ANALYSIS

#
==================================================
==========================
```

    - Comments the phase
  - SemanticAnalyzer Class
    - Initialization

```
class SemanticAnalyzer:
    def __init__(self):
            self.declared = set()     #
variables that have been declared
```

      - This initializes the semantic analyzer and creates a set named declared to keep track of all variables that have been declared in the program. This structure ensures that each variable is unique and prevents duplication.
    - Error Handling

```
def error(self, msg, line):
                                raise
SemanticError(f"[Semantic] Line {line}:
{msg}")
```

      - This function is used to raise semantic errors when violations occur. It includes the line number and a descriptive message.
    - Main Analysis Function

```
    def analyze(self, stmts):
```

```
        for stmt in stmts:
            self.check_stmt(stmt)
```

- This function serves as the entry point for semantic analysis. It iterates through all statements in the AST and passes each one to the check_stmt function for validation.

- Statement Checking Function

```
def check_stmt(self, stmt):
                    if  isinstance(stmt,
VarDeclNode):
                        if  stmt.name  in
self.declared:
                    self.error(f"Variable
'{stmt.name}'     already     declared",
stmt.line)

self.declared.add(stmt.name)

                  elif  isinstance(stmt,
AssignNode):
                    if  stmt.name  not  in
self.declared:
                    self.error(f"Variable
'{stmt.name}' used before declaration",
stmt.line)
            self.check_expr(stmt.expr)

                  elif  isinstance(stmt,
InputNode):
                    if  stmt.name  not  in
self.declared:
                    self.error(f"Variable
'{stmt.name}' used before declaration",
stmt.line)

                  elif  isinstance(stmt,
OutputNode):
                    if  stmt.name  not  in
self.declared:
                    self.error(f"Variable
'{stmt.name}' used before declaration",
```

```
stmt.line)
```

- This function is responsible for validating each statement in the program to ensure that it follows the semantic rules of the language. It first checks if a statement is a variable declaration and prevents redeclaration by verifying if the variable already exists in the declared set before adding it. For assignment statements, it ensures that the variable being assigned has already been declared and then validates the expression being assigned. For input and output statements, it verifies that the variable being used has been declared beforehand.

- Expression Checking Function

```
def check_expr(self, node):
                    if  isinstance(node,
NumberNode):
            return
        if isinstance(node, IdentNode):
                    if  node.name  not  in
self.declared:
                    self.error(f"Variable
'{node.name}' used before declaration",
node.line)
        if isinstance(node, BinOpNode):
            self.check_expr(node.left)
            self.check_expr(node.right)
```

- This function validates expressions to ensure that all values and variables used are semantically correct. It allows numeric values (NumberNode) without further checks since they are always valid. For identifiers (IdentNode), it verifies that the variable has already

been declared; otherwise, it raises an error to prevent the use of undeclared variables. If the node is a binary operation (BinOpNode), the function recursively checks both the left and right operands to ensure that all parts of the expression are valid. Overall, this function enforces proper variable usage within expressions and ensures that the program does not reference undeclared identifiers.

- Phase 4: Interpretation (Tree-Walk Execution)
  - Comment Block

```
#
==================================================
==========================
# PHASE 4: INTERPRETER (Tree-Walk Execution)
#
==================================================
==========================
```

    - Comments the phase
  - Interpreter Class
    - Initialization

```
class Interpreter:
    def __init__(self):
        self.env = {}    # variable name
→ integer value
```

      - This initializes the interpreter and creates an environment dictionary (env) that stores variable names as keys and their corresponding integer values as values. This serves as the runtime memory of the program.
    - Error Handling

```
    def error(self, msg):
        raise RuntimeError_(f"[Runtime]
{msg}")
```

- This function raises runtime errors during program execution. It provides a clear error message that helps identify issues such as invalid input or illegal operations.

- Main Execution Function

```
def run(self, stmts):
        for stmt in stmts:
            self.exec_stmt(stmt)
```

  - This function serves as the entry point for program execution. It iterates through all statements in the AST and executes each one using the exec_stmt function.

- Statement Execution Function

```
def exec_stmt(self, stmt):
                    if  isinstance(stmt,
VarDeclNode):
             self.env[stmt.name] = 0   #
default value

                  elif  isinstance(stmt,
AssignNode):
                   self.env[stmt.name]  =
self.eval_expr(stmt.expr)

                  elif  isinstance(stmt,
InputNode):
             raw = input(f"  Enter value
for '{stmt.name}': ")
            try:
                   self.env[stmt.name]  =
int(raw)
            except ValueError:
                   self.error(f"Expected
an integer, got {raw!r}")

                  elif  isinstance(stmt,
OutputNode):
                 print(f"   {stmt.name}  =
```

```
{self.env[stmt.name]}")
```

- This function is responsible for executing individual statements in the program based on their type during the interpretation phase. For variable declarations (VarDeclNode), it initializes the variable in the environment with a default value of 0. For assignment statements (AssignNode), it evaluates the expression on the right-hand side using eval_expr and stores the result in the corresponding variable. For input statements (InputNode), it prompts the user to enter a value, converts the input into an integer, and stores it in the environment, raising an error if the input is not a valid integer. For output statements (OutputNode), it retrieves the value of the specified variable from the environment and prints it to the user.

- Expression Evaluation Function

```
  def eval_expr(self, node):
                    if   isinstance(node,
NumberNode):
            return node.value

        if isinstance(node, IdentNode):
            return self.env[node.name]

        if isinstance(node, BinOpNode):
                                    lv   =
self.eval_expr(node.left)
                                    rv   =
self.eval_expr(node.right)
              if node.op == '+': return
lv + rv
```

```
                if node.op == '-': return
lv - rv
                if node.op == '*': return
lv * rv
            if node.op == '/':
                if rv == 0:

self.error("Division by zero")
                    return lv // rv    #
integer division
```

- This function is responsible for evaluating expressions in the program and returning their computed integer values during execution. If the node is a NumberNode, it simply returns the numeric value. If it is an IdentNode, it retrieves and returns the value of the variable from the environment (env). For BinOpNode, the function recursively evaluates both the left and right operands, then applies the შესაბამის arithmetic operation based on the operator (+, -, *, /). In the case of division, it includes a safety check to prevent division by zero by raising a runtime error if the right operand is zero, and it performs integer division using the floor division operator (//). Overall, this function enables the interpreter to compute the results of expressions correctly while handling basic errors during evaluation.

- Unknown Node Handling

```
        self.error(f"Unknown node type:
{type(node)}")
```

- This ensures that only recognized node

types are processed. If an unrecognized node is encountered, a runtime error is raised.

- Driver
  - Comment Block

```
#
==========================================================================
# DRIVER
#
==========================================================================
```

    - Comments the phase.
  - Header Formatting Function

```
def print_header(title, char='=', width=56):
    print(char * width)
    print(f"  {title}")
    print(char * width)
```

    - This function is used to display section headers in the console output. It prints a title centered between horizontal lines made of repeated characters, improving readability and visually separating different phases of the compiler process.
  - Section Formatting Function

```
def print_section(title):
    print(f"\n{'—' * 56}")
    print(f"  {title}")
    print(f"{'—' * 56}")
```

    - This function is used to display section headers in the console output. It prints a title centered between horizontal lines made of repeated characters, improving readability and visually separating different phases of the compiler process.
  - Compilation and Run Function

```
def compile_and_run(source, filepath):
```

```
    print_header("CSTPLANGS · Translation of Programming Language")
    print(f"  File : {filepath}")
    print(f"  Lines: {source.count(chr(10)) + 1}")

    # ── Source Listing ──────────────────────────────
    print_section("SOURCE CODE")
    for i, line in enumerate(source.splitlines(), 1):
        print(f"  {i:>3} │ {line}")

    # ── Phase 1: Lexing ─────────────────────────────
    print_section("PHASE 1 · LEXER — Tokenizing source")
    try:
        lexer  = Lexer(source)
        tokens = lexer.tokenize()
    except LexerError as e:
        print(f"\n  ✖ {e}")
        return

    # Pretty token table
    print(f"\n  {'#':<5} {'TYPE':<14} {'VALUE':<20} {'LINE'}")
    print(f"  {'─'*5} {'─'*14} {'─'*20} {'─'*4}")
    for i, tok in enumerate(tokens):
        if tok.type == TT_EOF:
            break
        print(f"  {i:<5} {tok.type:<14} {str(tok.value):<20} {tok.line}")
    print(f"\n  ✔ {len(tokens)-1} token(s) produced.")

    # ── Phase 2: Parsing ────────────────────────────
    print_section("PHASE 2 · PARSER — Building AST")
    try:
        parser = Parser(tokens)
```

```
        ast    = parser.parse()
    except ParseError as e:
        print(f"\n  ✖ {e}")
        return

    # AST summary
            print(f"\n      {'NODE  TYPE':<22}
{'DETAILS'}")
    print(f"  {'─'*22} {'─'*30}")
    for node in ast:
        if isinstance(node, VarDeclNode):
            print(f"  {'VarDeclNode':<22} var
{node.name}  (line {node.line})")
        elif isinstance(node, AssignNode):
                print(f"   {'AssignNode':<22}
{node.name} = <expr>  (line {node.line})")
        elif isinstance(node, InputNode):
            print(f"  {'InputNode':<22} input
{node.name}  (line {node.line})")
        elif isinstance(node, OutputNode):
                print(f"   {'OutputNode':<22}
output {node.name}  (line {node.line})")
      print(f"\n   ✔ {len(ast)} AST node(s)
produced.")

        # ── Phase 3: Semantic Analysis
──────────────────────────────
        print_section("PHASE 3 · SEMANTIC
ANALYSIS — Validating program")
    try:
        analyzer = SemanticAnalyzer()
        analyzer.analyze(ast)
    except SemanticError as e:
        print(f"\n  ✖ {e}")
        return

        print(f"\n    Declared variables :
{sorted(analyzer.declared)}")
    print(f"  ✔ No semantic errors found.")

            # ── Phase 4: Execution
──────────────────────────────────
     print_section("PHASE 4 · INTERPRETER —
```

```
Executing program")
    print()
    try:
        interpreter = Interpreter()
        interpreter.run(ast)
    except RuntimeError_ as e:
        print(f"\n  ✖ {e}")

        return

     print_section("DONE  —  Program finished
successfully")
    print()
```

- Coordinates the full compilation and execution process by displaying the source code, running lexical analysis, parsing, semantic checking, and interpretation in sequence, while handling errors and producing structured output for each phase.

- Main Driver Function

```
ef main():
    if len(sys.argv) < 2:
            print("Usage: python compiler.py
<source_file.txt>")
           print("Example: python compiler.py
program.txt")
        sys.exit(1)

    filepath = sys.argv[1]
    try:
        with open(filepath, 'r') as f:
            source = f.read()
    except FileNotFoundError:
         print(f"Error: File '{filepath}' not
found.",)
        sys.exit(1)

    compile_and_run(source, filepath)
```

- This function serves as the program's entry point by handling command-line input,

validating the provided file, reading the source code, and passing it to the compile-and-run pipeline while managing basic file-related errors.

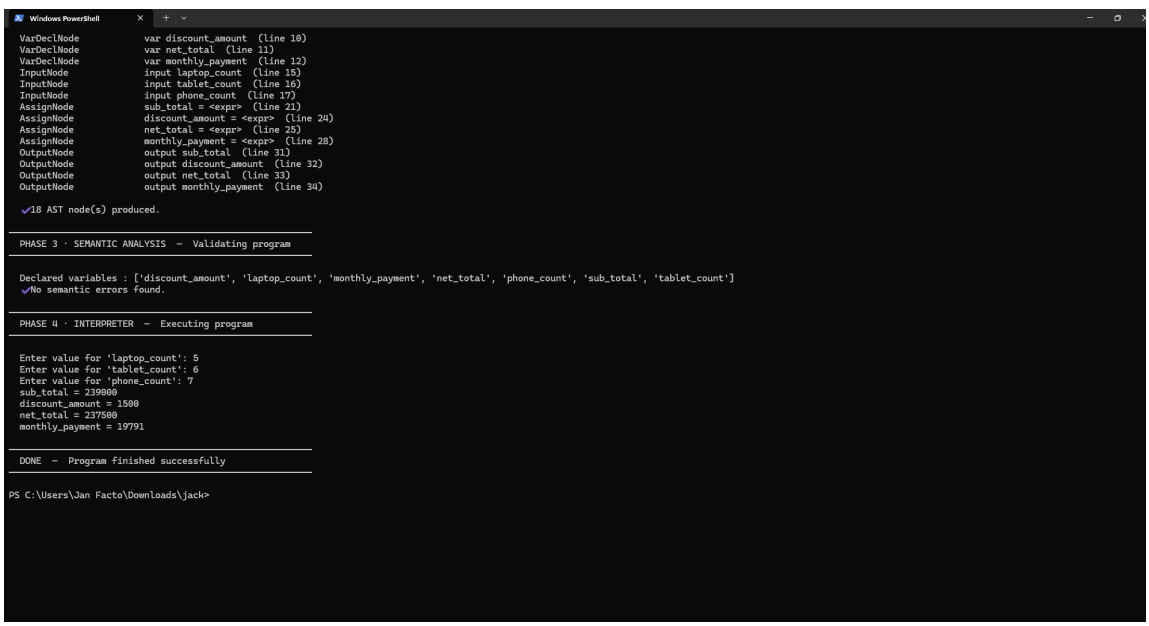
- Program Entry Point

```
if __name__ == '__main__':
    main()
```

  - Ensures that the main() function is executed only when the script is run directly, not when it is imported as a module, thereby controlling the program's execution flow.

IV. Use-Cases

The system was validated through two primary test scenarios to ensure robust error handling and successful execution.

- **Successful Execution (Valid Script):** Using jack_language.txt, the system successfully initializes shop inventory, calculates a sub-total of electronics, applies a 1,500-unit discount, and divides the remainder into a 12-month payment plan without errors.
- 

- **Error Handling (Invalid Script):** Using jack_wrong.txt, the system demonstrated its defensive programming capabilities by identifying a missing semicolon on Line 6, halting the process before any faulty data could be calculated.

```
Windows PowerShell
 35    RPAREN        )                  21
 36    PLUS          +                  21
 37    LPAREN        (                  21
 38    IDENTIFIER    tablet_count       21
 39    MUL           *                  21
 40    INTEGER       10000              21
 41    RPAREN        )                  21
 42    PLUS          +                  21
 43    LPAREN        (                  21
 44    IDENTIFIER    phone_count        21
 45    MUL           *                  21
 46    INTEGER       7000               21
 47    RPAREN        )                  21
 48    SEMICOLON     ;                  21
 49    IDENTIFIER    discount_amount    24
 50    ASSIGN        =                  24
 51    INTEGER       1500               24
 52    SEMICOLON     ;                  24
 53    IDENTIFIER    net_total          25
 54    ASSIGN        =                  25
 55    IDENTIFIER    sub_total          25
 56    MINUS         -                  25
 57    IDENTIFIER    discount_amount    25
 58    SEMICOLON     ;                  25
 59    IDENTIFIER    monthly_payment    28
 60    ASSIGN        =                  28
 61    IDENTIFIER    net_total          28
 62    DIV           /                  28
 63    INTEGER       12                 28
 64    SEMICOLON     ;                  28
 65    OUTPUT        output             31
 66    IDENTIFIER    sub_total          31
 67    SEMICOLON     ;                  31
 68    OUTPUT        output             32
 69    IDENTIFIER    discount_amount    32
 70    SEMICOLON     ;                  32
 71    OUTPUT        output             33
 72    IDENTIFIER    net_total          33
 73    SEMICOLON     ;                  33
 74    OUTPUT        output             34
 75    IDENTIFIER    monthly_payment    34
 76    SEMICOLON     ;                  34

 ✓77 token(s) produced.

 PHASE 2 · PARSER  –  Building AST

 ✗[Parser] Line 7: Expected SEMICOLON, got VAR ('var')
PS C:\Users\Jan Facto\Downloads\jack>
```

V. Task Delegations

- **Coleen Raye C. Magbag**
  - **Lexical Analyzer**
    - Create the lexer in Python that reads jack_language.txt, removes comments and whitespace, identifies and classifies tokens, tracks line numbers, and displays the token table.
- **Aislinn H. Uy**
  - **Syntax Analyzer**
    - Create the syntax analyzer in Python that defines and implements the grammar rules, building the parser using recursive descent, and constructing the Abstract Syntax Tree (AST) while ensuring correct syntax structure.
- **Kaiser G. Opulencia**
  - **Code Interpreter**
    - Create the implementation of the interpreter and expression evaluation, variable management, and program execution, as well as designing and testing the system's use cases to demonstrate valid execution, syntax errors, and logic errors.
- **Janelle P. Facto**
  - **Documentation and Driver**
    - Create the system documentation and develop the driver program

that integrates all phases of the compiler, and present the final structured output of the system.

VI. Source Code

- **jack_language.txt**

```
/* jack_language.txt */

/* JanTayoBibili Shop Billing System */

/* Variable Declarations */
var laptop_count;
var tablet_count;
var phone_count;
var sub_total;
var discount_amount;
var net_total;
var monthly_payment;

/* Input Quantity from User */
input laptop_count;
input tablet_count;
input phone_count;

/* Sub-total Calculation (Multiplication and Addition) */
/* Prices: Laptop = 26000, Tablet = 10000, Phone = 7000 */
sub_total = (laptop_count * 26000) + (tablet_count * 10000) + (phone_count
* 7000);

/* Apply Fixed Discount (Subtraction) */
discount_amount = 1500;
net_total = sub_total - discount_amount;

/* Calculate 12-Month Installment (Division) */
monthly_payment = net_total / 12;

/* Display results */
output sub_total;
output discount_amount;
output net_total;
output monthly_payment;
```

- **jack_wrong.txt**

```
/* jack_language.txt */
```

```
/* JanTayoBibili Shop Billing System */

/* Variable Declarations */
var laptop_count
var tablet_count;
var phone_count;
var sub_total;
var discount_amount;
var net_total;
var monthly_payment;

/* Input Quantity from User */
input laptop_count;
input tablet_count;
input phone_count;

/* Sub-total Calculation (Multiplication and Addition) */
/* Prices: Laptop = 26000, Tablet = 10000, Phone = 7000 */
sub_total = (laptop_count * 26000) + (tablet_count * 10000) + (phone_count
* 7000);

/* Apply Fixed Discount (Subtraction) */
discount_amount = 1500;
net_total = sub_total - discount_amount;

/* Calculate 12-Month Installment (Division) */
monthly_payment = net_total / 12;

/* Display results */
output sub_total;
output discount_amount;
output net_total;
output monthly_payment;
```

- **Jack.py**

```
"""
CSTPLANGS - Simple Language Compiler/Interpreter
FEU Institute of Technology - Computer Science Department

Phases:
  1. Lexer       - Tokenizes input source code
  2. Parser      - Builds an Abstract Syntax Tree (AST)
```

```
  3. Semantic    - Type checks and validates declarations
  4. Interpreter - Executes the AST
"""

import sys
import emoji

#
==============================================================================
===
# PHASE 1: LEXER (Tokenizer)
#
==============================================================================
===

# Token types
TT_VAR    = 'VAR'
TT_INPUT  = 'INPUT'
TT_OUTPUT = 'OUTPUT'
TT_IDENT  = 'IDENTIFIER'
TT_INT    = 'INTEGER'
TT_PLUS   = 'PLUS'
TT_MINUS  = 'MINUS'
TT_MUL    = 'MUL'
TT_DIV    = 'DIV'
TT_ASSIGN = 'ASSIGN'
TT_SEMI   = 'SEMICOLON'
TT_LPAREN = 'LPAREN'
TT_RPAREN = 'RPAREN'
TT_EOF    = 'EOF'

KEYWORDS = {'var', 'input', 'output'}


class Token:
    def __init__(self, type_, value, line):
        self.type  = type_
        self.value = value
        self.line  = line

    def __repr__(self):
        return f'Token({self.type}, {self.value!r}, line={self.line})'
```

```python
class LexerError(Exception):
    pass

class ParseError(Exception):
    pass

class SemanticError(Exception):
    pass

class RuntimeError_(Exception):
    pass


class Lexer:
    def __init__(self, source):
        self.source = source
        self.pos    = 0
        self.line   = 1
        self.tokens = []

    def error(self, msg):
        raise LexerError(f"[Lexer] Line {self.line}: {msg}")


    def peek(self):
        return self.source[self.pos] if self.pos < len(self.source) else
None

    def advance(self):
        ch = self.source[self.pos]
        self.pos += 1
        if ch == '\n':
            self.line += 1
        return ch

    def skip_whitespace(self):
        while self.peek() and self.peek() in ' \t\r\n':
            self.advance()

    def skip_comment(self):
        """Skip /* ... */ block comments (may span multiple lines)."""
        self.advance()  # consume '/'
```

```
        self.advance()  # consume '*'
        while self.pos < len(self.source):
            if self.source[self.pos] == '*' and self.pos + 1 <
len(self.source) and self.source[self.pos + 1] == '/':
                self.advance()  # consume '*'
                self.advance()  # consume '/'
                return
            self.advance()
        self.error("Unterminated block comment")

    def read_number(self):
        start = self.pos
        while self.peek() and self.peek().isdigit():
            self.advance()
        return self.source[start:self.pos]

    def read_identifier(self):
        start = self.pos
        while self.peek() and (self.peek().isalnum() or self.peek() ==
'_'):
            self.advance()
        return self.source[start:self.pos]

    def tokenize(self):
        simple = {
            '+': TT_PLUS,
            '-': TT_MINUS,
            '*': TT_MUL,
            '/': TT_DIV,
            '=': TT_ASSIGN,
            ';': TT_SEMI,
            '(': TT_LPAREN,
            ')': TT_RPAREN,
        }

        while self.pos < len(self.source):
            self.skip_whitespace()
            if self.pos >= len(self.source):
                break

            ch = self.peek()

            # Block comment
```

```
            if ch == '/' and self.pos + 1 < len(self.source) and
self.source[self.pos + 1] == '*':
                self.skip_comment()
                continue

            # Integer literal
            if ch.isdigit():
                line = self.line
                num  = self.read_number()
                self.tokens.append(Token(TT_INT, int(num), line))
                continue

            # Identifier or keyword
            if ch.isalpha() or ch == '_':
                line  = self.line
                ident = self.read_identifier()
                if ident in KEYWORDS:
                    tt = {'var': TT_VAR, 'input': TT_INPUT, 'output':
TT_OUTPUT}[ident]
                    self.tokens.append(Token(tt, ident, line))
                else:
                    self.tokens.append(Token(TT_IDENT, ident, line))
                continue

            # Single-character tokens
            if ch in simple:
                line = self.line
                self.advance()
                self.tokens.append(Token(simple[ch], ch, line))
                continue

            self.error(f"Unexpected character: {ch!r}")

        self.tokens.append(Token(TT_EOF, None, self.line))
        return self.tokens


#
==========================================================================
===
# PHASE 2: PARSER  (Recursive-Descent → AST)
#
==========================================================================
```

```
===

# AST node classes
class VarDeclNode:
    """var <name> ;"""
    def __init__(self, name, line):
        self.name = name
        self.line = line

class AssignNode:
    """<name> = <expr> ;"""
    def __init__(self, name, expr, line):
        self.name = name
        self.expr = expr
        self.line = line

class InputNode:
    """input <name> ;"""
    def __init__(self, name, line):
        self.name = name
        self.line = line

class OutputNode:
    """output <name> ;"""
    def __init__(self, name, line):
        self.name = name
        self.line = line

class BinOpNode:
    """<left> op <right>"""
    def __init__(self, left, op, right):
        self.left  = left
        self.op    = op
        self.right = right

class NumberNode:
    def __init__(self, value):
        self.value = value

class IdentNode:
    def __init__(self, name, line):
        self.name = name
        self.line = line
```

```python
class Parser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.pos    = 0

    def current(self):
        return self.tokens[self.pos]

    def error(self, msg):
        tok = self.current()
        raise ParseError(f"[Parser] Line {tok.line}: {msg}")

    def eat(self, type_):
        tok = self.current()
        if tok.type != type_:
            self.error(f"Expected {type_}, got {tok.type}
({tok.value!r})")
        self.pos += 1
        return tok

    def peek_type(self):
        return self.current().type

    # --- Grammar rules ---

    def parse(self):
        """program → statement* EOF"""
        stmts = []
        while self.peek_type() != TT_EOF:
            stmts.append(self.statement())
        return stmts

    def statement(self):
        tt = self.peek_type()
        if tt == TT_VAR:
            return self.var_decl()
        if tt == TT_INPUT:
            return self.input_stmt()
        if tt == TT_OUTPUT:
            return self.output_stmt()
        if tt == TT_IDENT:
```

```
            return self.assign_stmt()
        self.error(f"Unexpected token: {self.current().value!r}")

    def var_decl(self):
        """var <ident> ;"""
        line = self.current().line
        self.eat(TT_VAR)
        name_tok = self.eat(TT_IDENT)
        self.eat(TT_SEMI)
        return VarDeclNode(name_tok.value, line)

    def input_stmt(self):
        """input <ident> ;"""
        line = self.current().line
        self.eat(TT_INPUT)
        name_tok = self.eat(TT_IDENT)
        self.eat(TT_SEMI)
        return InputNode(name_tok.value, line)

    def output_stmt(self):
        """output <ident> ;"""
        line = self.current().line
        self.eat(TT_OUTPUT)
        name_tok = self.eat(TT_IDENT)
        self.eat(TT_SEMI)
        return OutputNode(name_tok.value, line)

    def assign_stmt(self):
        """<ident> = <expr> ;"""
        line = self.current().line
        name_tok = self.eat(TT_IDENT)
        self.eat(TT_ASSIGN)
        expr = self.expr()
        self.eat(TT_SEMI)
        return AssignNode(name_tok.value, expr, line)

    # --- Expression parsing (operator precedence) ---
    #  expr   → term   (('+' | '-') term)*
    #  term   → factor (('*' | '/') factor)*
    #  factor → NUMBER | IDENT | '(' expr ')'

    def expr(self):
        node = self.term()
```

```
        while self.peek_type() in (TT_PLUS, TT_MINUS):
            op  = self.eat(self.peek_type())
            rhs = self.term()
            node = BinOpNode(node, op.value, rhs)
        return node

    def term(self):
        node = self.factor()
        while self.peek_type() in (TT_MUL, TT_DIV):
            op  = self.eat(self.peek_type())
            rhs = self.factor()
            node = BinOpNode(node, op.value, rhs)
        return node

    def factor(self):
        tok = self.current()
        if tok.type == TT_INT:
            self.eat(TT_INT)
            return NumberNode(tok.value)
        if tok.type == TT_IDENT:
            self.eat(TT_IDENT)
            return IdentNode(tok.value, tok.line)
        if tok.type == TT_LPAREN:
            self.eat(TT_LPAREN)
            node = self.expr()
            self.eat(TT_RPAREN)
            return node
        self.error(f"Expected number, identifier, or '(' — got
{tok.value!r}")


#
==========================================================================
===
# PHASE 3: SEMANTIC ANALYSIS
#
==========================================================================
===

class SemanticAnalyzer:
    def __init__(self):
        self.declared = set()   # variables that have been declared
```

```
    def error(self, msg, line):
        raise SemanticError(f"[Semantic] Line {line}: {msg}")

    def analyze(self, stmts):
        for stmt in stmts:
            self.check_stmt(stmt)

    def check_stmt(self, stmt):
        if isinstance(stmt, VarDeclNode):
            if stmt.name in self.declared:
                self.error(f"Variable '{stmt.name}' already declared",
stmt.line)
            self.declared.add(stmt.name)

        elif isinstance(stmt, AssignNode):
            if stmt.name not in self.declared:
                self.error(f"Variable '{stmt.name}' used before
declaration", stmt.line)
            self.check_expr(stmt.expr)

        elif isinstance(stmt, InputNode):
            if stmt.name not in self.declared:
                self.error(f"Variable '{stmt.name}' used before
declaration", stmt.line)

        elif isinstance(stmt, OutputNode):
            if stmt.name not in self.declared:
                self.error(f"Variable '{stmt.name}' used before
declaration", stmt.line)

    def check_expr(self, node):
        if isinstance(node, NumberNode):
            return
        if isinstance(node, IdentNode):
            if node.name not in self.declared:
                self.error(f"Variable '{node.name}' used before
declaration", node.line)
        if isinstance(node, BinOpNode):
            self.check_expr(node.left)
            self.check_expr(node.right)


#
```

```
================================================================================
# PHASE 4: INTERPRETER (Tree-Walk Execution)
#
================================================================================

class Interpreter:
    def __init__(self):
        self.env = {}   # variable name → integer value

    def error(self, msg):
        raise RuntimeError_(f"[Runtime] {msg}")

    def run(self, stmts):
        for stmt in stmts:
            self.exec_stmt(stmt)

    def exec_stmt(self, stmt):
        if isinstance(stmt, VarDeclNode):
            self.env[stmt.name] = 0   # default value

        elif isinstance(stmt, AssignNode):
            self.env[stmt.name] = self.eval_expr(stmt.expr)

        elif isinstance(stmt, InputNode):
            raw = input(f"  Enter value for '{stmt.name}': ")
            try:
                self.env[stmt.name] = int(raw)
            except ValueError:
                self.error(f"Expected an integer, got {raw!r}")

        elif isinstance(stmt, OutputNode):
            print(f"  {stmt.name} = {self.env[stmt.name]}")

    def eval_expr(self, node):
        if isinstance(node, NumberNode):
            return node.value

        if isinstance(node, IdentNode):
            return self.env[node.name]

        if isinstance(node, BinOpNode):
```

```
            lv = self.eval_expr(node.left)
            rv = self.eval_expr(node.right)
            if node.op == '+': return lv + rv
            if node.op == '-': return lv - rv
            if node.op == '*': return lv * rv
            if node.op == '/':
                if rv == 0:
                    self.error("Division by zero")
                return lv // rv   # integer division

        self.error(f"Unknown node type: {type(node)}")


#
==========================================================================
===
# DRIVER
#
==========================================================================
===

def print_header(title, char='=', width=56):
    print(char * width)
    print(f"  {title}")
    print(char * width)


def print_section(title):
    print(f"\n{'—' * 56}")
    print(f"  {title}")
    print(f"{'—' * 56}")


def compile_and_run(source, filepath):
    print_header("CSTPLANGS  ·  Translation of Programming Language")
    print(f"  File : {filepath}")
    print(f"  Lines: {source.count(chr(10)) + 1}")

    # ── Source Listing ──────────────────────────────────────
    print_section("SOURCE CODE")
    for i, line in enumerate(source.splitlines(), 1):
        print(f"  {i:>3} │  {line}")
```

```
    # ── Phase 1: Lexing ──────────────────────────────────────
    print_section("PHASE 1 · LEXER  —  Tokenizing source")
    try:
        lexer  = Lexer(source)
        tokens = lexer.tokenize()
    except LexerError as e:
        print(f"\n  ✖ {e}")
        return

    # Pretty token table
    print(f"\n  {'#':<5} {'TYPE':<14} {'VALUE':<20} {'LINE'}")
    print(f"  {'─'*5} {'─'*14} {'─'*20} {'─'*4}")
    for i, tok in enumerate(tokens):
        if tok.type == TT_EOF:
            break
        print(f"  {i:<5} {tok.type:<14} {str(tok.value):<20} {tok.line}")
    print(f"\n  ✔ {len(tokens)-1} token(s) produced.")

    # ── Phase 2: Parsing ─────────────────────────────────────
    print_section("PHASE 2 · PARSER  —  Building AST")
    try:
        parser = Parser(tokens)
        ast    = parser.parse()
    except ParseError as e:
        print(f"\n  ✖ {e}")
        return

    # AST summary
    print(f"\n  {'NODE TYPE':<22} {'DETAILS'}")
    print(f"  {'─'*22} {'─'*30}")
    for node in ast:
        if isinstance(node, VarDeclNode):
            print(f"  {'VarDeclNode':<22} var {node.name}  (line
{node.line})")
        elif isinstance(node, AssignNode):
            print(f"  {'AssignNode':<22} {node.name} = <expr>  (line
{node.line})")
        elif isinstance(node, InputNode):
            print(f"  {'InputNode':<22} input {node.name}  (line
{node.line})")
        elif isinstance(node, OutputNode):
            print(f"  {'OutputNode':<22} output {node.name}  (line
{node.line})")
```

```
    print(f"\n  ✔ {len(ast)} AST node(s) produced.")

    # ── Phase 3: Semantic Analysis ──────────────────────────────
    print_section("PHASE 3 · SEMANTIC ANALYSIS  —  Validating program")
    try:
        analyzer = SemanticAnalyzer()
        analyzer.analyze(ast)
    except SemanticError as e:
        print(f"\n  ✖ {e}")
        return

    print(f"\n  Declared variables : {sorted(analyzer.declared)}")
    print(f"  ✔ No semantic errors found.")

    # ── Phase 4: Execution ──────────────────────────────────────
    print_section("PHASE 4 · INTERPRETER  —  Executing program")
    print()
    try:
        interpreter = Interpreter()
        interpreter.run(ast)
    except RuntimeError_ as e:
        print(f"\n  ✖ {e}")

        return

    print_section("DONE  —  Program finished successfully")
    print()


def main():
    if len(sys.argv) < 2:
        print("Usage: python compiler.py <source_file.txt>")
        print("Example: python compiler.py program.txt")
        sys.exit(1)

    filepath = sys.argv[1]
    try:
        with open(filepath, 'r') as f:
            source = f.read()
    except FileNotFoundError:
        print(f"Error: File '{filepath}' not found.",)
        sys.exit(1)
```

```python
    compile_and_run(source, filepath)



if __name__ == '__main__':
    main()
```

Screenshots of Outputs:

- **Using jack_language.txt (Correct File)**

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Jan Facto> cd "C:\Users\Jan Facto\Downloads\jack"
PS C:\Users\Jan Facto\Downloads\jack> python JACK.py jack_language.txt
=====================================================
  CSTPLANGS  ·  Translation of Programming Language
=====================================================
  File : jack_language.txt
  Lines: 35

  ─────────────────────────────────────────────────────
  SOURCE CODE
  ─────────────────────────────────────────────────────
    1 │ /* jack_language.txt */
    2 │
    3 │ /* JanTayoBibili Shop Billing System */
    4 │
    5 │ /* Variable Declarations */
    6 │ var laptop_count;
    7 │ var tablet_count;
    8 │ var phone_count;
    9 │ var sub_total;
   10 │ var discount_amount;
   11 │ var net_total;
   12 │ var monthly_payment;
   13 │
   14 │ /* Input Quantity from User */
   15 │ input laptop_count;
   16 │ input tablet_count;
   17 │ input phone_count;
   18 │
   19 │ /* Sub-total Calculation (Multiplication and Addition) */
   20 │ /* Prices: Laptop = 26000, Tablet = 10000, Phone = 7000 */
   21 │ sub_total = (laptop_count * 26000) + (tablet_count * 10000) + (phone_count * 7000);
   22 │
   23 │ /* Apply Fixed Discount (Subtraction) */
   24 │ discount_amount = 1500;
   25 │ net_total = sub_total - discount_amount;
   26 │
   27 │ /* Calculate 12-Month Installment (Division) */
   28 │ monthly_payment = net_total / 12;
   29 │
   30 │ /* Display results */
   31 │ output sub_total;
   32 │ output discount_amount;
   33 │ output net_total;
   34 │ output monthly_payment;
```

```
Windows PowerShell

  ─────────────────────────────────────────────────────
  PHASE 1 · LEXER  —  Tokenizing source
  ─────────────────────────────────────────────────────

  #    TYPE           VALUE                LINE
  ───  ─────────────  ───────────────────  ────
  0    VAR            var                  6
  1    IDENTIFIER     laptop_count         6
  2    SEMICOLON      ;                    6
  3    VAR            var                  7
  4    IDENTIFIER     tablet_count         7
  5    SEMICOLON      ;                    7
  6    VAR            var                  8
  7    IDENTIFIER     phone_count          8
  8    SEMICOLON      ;                    8
  9    VAR            var                  9
  10   IDENTIFIER     sub_total            9
  11   SEMICOLON      ;                    9
  12   VAR            var                  10
  13   IDENTIFIER     discount_amount      10
  14   SEMICOLON      ;                    10
  15   VAR            var                  11
  16   IDENTIFIER     net_total            11
  17   SEMICOLON      ;                    11
  18   VAR            var                  12
  19   IDENTIFIER     monthly_payment      12
  20   SEMICOLON      ;                    12
  21   INPUT          input                15
  22   IDENTIFIER     laptop_count         15
  23   SEMICOLON      ;                    15
  24   INPUT          input                16
  25   IDENTIFIER     tablet_count         16
  26   SEMICOLON      ;                    16
  27   INPUT          input                17
  28   IDENTIFIER     phone_count          17
  29   SEMICOLON      ;                    17
  30   IDENTIFIER     sub_total            21
  31   ASSIGN         =                    21
  32   LPAREN         (                    21
  33   IDENTIFIER     laptop_count         21
  34   MUL            *                    21
  35   INTEGER        26000                21
  36   RPAREN         )                    21
  37   PLUS           +                    21
  38   LPAREN         (                    21
  39   IDENTIFIER     tablet_count         21
  40   MUL            *                    21
  41   INTEGER        10000                21
  42   RPAREN         )                    21
  43   PLUS           +                    21
  44   LPAREN         (                    21
```

```
Windows PowerShell
45    IDENTIFIER     phone_count          21
46    MUL            *                    21
47    INTEGER        7000                 21
48    RPAREN         )                    21
49    SEMICOLON      ;                    21
50    IDENTIFIER     discount_amount      24
51    ASSIGN         =                    24
52    INTEGER        1500                 24
53    SEMICOLON      ;                    24
54    IDENTIFIER     net_total            25
55    ASSIGN         =                    25
56    IDENTIFIER     sub_total            25
57    MINUS          -                    25
58    IDENTIFIER     discount_amount      25
59    SEMICOLON      ;                    25
60    IDENTIFIER     monthly_payment      28
61    ASSIGN         =                    28
62    IDENTIFIER     net_total            28
63    DIV            /                    28
64    INTEGER        12                   28
65    SEMICOLON      ;                    28
66    OUTPUT         output               31
67    IDENTIFIER     sub_total            31
68    SEMICOLON      ;                    31
69    OUTPUT         output               32
70    IDENTIFIER     discount_amount      32
71    SEMICOLON      ;                    32
72    OUTPUT         output               33
73    IDENTIFIER     net_total            33
74    SEMICOLON      ;                    33
75    OUTPUT         output               34
76    IDENTIFIER     monthly_payment      34
77    SEMICOLON      ;                    34

✔78 token(s) produced.

PHASE 2 · PARSER  —  Building AST

NODE TYPE          DETAILS
VarDeclNode        var laptop_count  (line 6)
VarDeclNode        var tablet_count  (line 7)
VarDeclNode        var phone_count  (line 8)
VarDeclNode        var sub_total  (line 9)
VarDeclNode        var discount_amount  (line 10)
VarDeclNode        var net_total  (line 11)
VarDeclNode        var monthly_payment  (line 12)
InputNode          input laptop_count  (line 15)
InputNode          input tablet_count  (line 16)
```

```
Windows PowerShell
VarDeclNode        var discount_amount  (line 10)
VarDeclNode        var net_total  (line 11)
VarDeclNode        var monthly_payment  (line 12)
InputNode          input laptop_count  (line 15)
InputNode          input tablet_count  (line 16)
InputNode          input phone_count  (line 17)
AssignNode         sub_total = <expr>  (line 21)
AssignNode         discount_amount = <expr>  (line 24)
AssignNode         net_total = <expr>  (line 25)
AssignNode         monthly_payment = <expr>  (line 28)
OutputNode         output sub_total  (line 31)
OutputNode         output discount_amount  (line 32)
OutputNode         output net_total  (line 33)
OutputNode         output monthly_payment  (line 34)

✔18 AST node(s) produced.

PHASE 3 · SEMANTIC ANALYSIS  —  Validating program

Declared variables : ['discount_amount', 'laptop_count', 'monthly_payment', 'net_total', 'phone_count', 'sub_total', 'tablet_count']
✔No semantic errors found.

PHASE 4 · INTERPRETER  —  Executing program

Enter value for 'laptop_count': 5
Enter value for 'tablet_count': 6
Enter value for 'phone_count': 7
sub_total = 239000
discount_amount = 1500
net_total = 237500
monthly_payment = 19791

DONE  —  Program finished successfully

PS C:\Users\Jan Facto\Downloads\jack>
```

- **Using jack_wrong.txt (Incorrect File)**

```
Windows PowerShell

PS C:\Users\Jan Facto\Downloads\jack> python JACK.py jack_wrong.txt
==========================================================
  CSTPLANGS  ·  Translation of Programming Language
==========================================================
  File : jack_wrong.txt
  Lines: 35

──────────────────────────────────────────────────────────
  SOURCE CODE
──────────────────────────────────────────────────────────
   1 │  /* jack_language.txt */
   2 │
   3 │  /* JanTayoBibili Shop Billing System */
   4 │
   5 │  /* Variable Declarations */
   6 │  var laptop_count
   7 │  var tablet_count;
   8 │  var phone_count;
   9 │  var sub_total;
  10 │  var discount_amount;
  11 │  var net_total;
  12 │  var monthly_payment;
  13 │
  14 │  /* Input Quantity from User */
  15 │  input laptop_count;
  16 │  input tablet_count;
  17 │  input phone_count;
  18 │
  19 │  /* Sub-total Calculation (Multiplication and Addition) */
  20 │  /* Prices: Laptop = 26000, Tablet = 10000, Phone = 7000 */
  21 │  sub_total = (laptop_count * 26000) + (tablet_count * 10000) + (phone_count * 7000);
  22 │
  23 │  /* Apply Fixed Discount (Subtraction) */
  24 │  discount_amount = 1500;
  25 │  net_total = sub_total - discount_amount;
  26 │
  27 │  /* Calculate 12-Month Installment (Division) */
  28 │  monthly_payment = net_total / 12;
  29 │
  30 │  /* Display results */
  31 │  output sub_total;
  32 │  output discount_amount;
  33 │  output net_total;
  34 │  output monthly_payment;

──────────────────────────────────────────────────────────
  PHASE 1 · LEXER  —  Tokenizing source
──────────────────────────────────────────────────────────

  #    TYPE           VALUE                LINE
```

```
Windows PowerShell

  ────  ──────────────  ────────────────────  ────
  0     VAR             var                   6
  1     IDENTIFIER      laptop_count          6
  2     VAR             var                   7
  3     IDENTIFIER      tablet_count          7
  4     SEMICOLON       ;                     7
  5     VAR             var                   8
  6     IDENTIFIER      phone_count           8
  7     SEMICOLON       ;                     8
  8     VAR             var                   9
  9     IDENTIFIER      sub_total             9
  10    SEMICOLON       ;                     9
  11    VAR             var                   10
  12    IDENTIFIER      discount_amount       10
  13    SEMICOLON       ;                     10
  14    VAR             var                   11
  15    IDENTIFIER      net_total             11
  16    SEMICOLON       ;                     11
  17    VAR             var                   12
  18    IDENTIFIER      monthly_payment       12
  19    SEMICOLON       ;                     12
  20    INPUT           input                 15
  21    IDENTIFIER      laptop_count          15
  22    SEMICOLON       ;                     15
  23    INPUT           input                 16
  24    IDENTIFIER      tablet_count          16
  25    SEMICOLON       ;                     16
  26    INPUT           input                 17
  27    IDENTIFIER      phone_count           17
  28    SEMICOLON       ;                     17
  29    IDENTIFIER      sub_total             21
  30    ASSIGN          =                     21
  31    LPAREN          (                     21
  32    IDENTIFIER      laptop_count          21
  33    MUL             *                     21
  34    INTEGER         26000                 21
  35    RPAREN          )                     21
  36    PLUS            +                     21
  37    LPAREN          (                     21
  38    IDENTIFIER      tablet_count          21
  39    MUL             *                     21
  40    INTEGER         10000                 21
  41    RPAREN          )                     21
  42    PLUS            +                     21
  43    LPAREN          (                     21
  44    IDENTIFIER      phone_count           21
  45    MUL             *                     21
  46    INTEGER         7000                  21
  47    RPAREN          )                     21
  48    SEMICOLON       ;                     21
  49    IDENTIFIER      discount_amount       24
```

```
Windows PowerShell
 35   RPAREN        )                  21
 36   PLUS          +                  21
 37   LPAREN        (                  21
 38   IDENTIFIER    tablet_count       21
 39   MUL           *                  21
 40   INTEGER       10000              21
 41   RPAREN        )                  21
 42   PLUS          +                  21
 43   LPAREN        (                  21
 44   IDENTIFIER    phone_count        21
 45   MUL           *                  21
 46   INTEGER       7000               21
 47   RPAREN        )                  21
 48   SEMICOLON     ;                  21
 49   IDENTIFIER    discount_amount    24
 50   ASSIGN        =                  24
 51   INTEGER       1500               24
 52   SEMICOLON     ;                  24
 53   IDENTIFIER    net_total          25
 54   ASSIGN        =                  25
 55   IDENTIFIER    sub_total          25
 56   MINUS         -                  25
 57   IDENTIFIER    discount_amount    25
 58   SEMICOLON     ;                  25
 59   IDENTIFIER    monthly_payment    28
 60   ASSIGN        =                  28
 61   IDENTIFIER    net_total          28
 62   DIV           /                  28
 63   INTEGER       12                 28
 64   SEMICOLON     ;                  28
 65   OUTPUT        output             31
 66   IDENTIFIER    sub_total          31
 67   SEMICOLON     ;                  31
 68   OUTPUT        output             32
 69   IDENTIFIER    discount_amount    32
 70   SEMICOLON     ;                  32
 71   OUTPUT        output             33
 72   IDENTIFIER    net_total          33
 73   SEMICOLON     ;                  33
 74   OUTPUT        output             34
 75   IDENTIFIER    monthly_payment    34
 76   SEMICOLON     ;                  34

 ✓77 token(s) produced.

 PHASE 2 · PARSER  —  Building AST

 ✗[Parser] Line 7: Expected SEMICOLON, got VAR ('var')
PS C:\Users\Jan Facto\Downloads\jack>
```

References:

- Computer Hope. (n.d.). Programming language. https://www.computerhope.com/jargon/p/programming-language.htm
- Chopra, R. (2015). Principles of programming languages. New Delhi: I.K. International Publishing.
- Kumar, S. (2015). Principles of programming languages. S. K. Kataria & Sons.
- McGrath, M. (2017). C++ programming. In Easy Steps Limited.
- Perkins, B. (2016). Beginning Visual C# 2015 programming. Wrox.
- TutorialsPoint. (n.d.). Compiler design: Syntax analysis. https://www.tutorialspoint.com/compiler_design/compiler_design_syntax_analysis.htm
- TutorialsPoint. (n.d.). Compiler design: Semantic analysis. https://www.tutorialspoint.com/compiler_design/compiler_design_semantic_analysis.htm
- LMU Computer Science. (n.d.). Control flow. https://cs.lmu.edu/~ray/notes/controlflow/
- University of Southern Indiana. (n.d.). Scope in programming languages. https://www.cs.iusb.edu/~dvrajito/teach/c311/c311_3_scope.html
- Teach Computer Science. (n.d.). Programming data types.

https://teachcomputerscience.com/programming-data-types/

- Princeton University. (n.d.). Algorithms, 4th edition (OOP). https://algs4.cs.princeton.edu/12oop/

ASSESSMENT:

Department	COMPUTER SCIENCE
Subject Code	CS0035
Description	PROGRAMMING LANGUAGES
Term/Academic Year	2nd Term SY 2025-2026

Topic	Lessons 1-9
Project	1
Project Activity	**Project Presentation**
CLO	**1, 2, 3**

Note: The following rubrics/metrics will be used to grade students' output in the project.

Criteria	Exceptional	Acceptable	Amateur	Unsatisfactory
Specifications (40%)	The project works and meets all of the specifications. **(31-40)**	The project works and produces the correct results and displays them correctly. It also meets most of the other specifications. **(21-30)**	The project produces correct results but does not display them correctly. **(11-20)**	The project is producing incorrect results. **(1-10)**
Design (15 %)	The design is exceptionally attractive. Project is "user-friendly" with informative and consistent prompts and messages. **(13-15)**	The design is fairly attractive. Project is "user-friendly" with informative and consistent prompts and messages. **(9-12)**	The design is fairly attractive. Project is not "user-friendly" but still provide informative and consistent prompts and messages. **(5-8)**	The design is unattractive and not user-friendly **(1-4)**
Efficiency (20%)	The code is extremely efficient without sacrificing readability and understanding. **(16-20)**	The code is fairly efficient without sacrificing readability and understanding. **(11-15)**	The code is brute force and unnecessarily long. **(6-10)**	The code is huge and appears to be patched together. **(1-5)**
Readability (10 %)	The code is exceptionally well organized and very easy to follow. **(8-10)**	The code is fairly easy to read. **(5-7)**	The code is readable only by someone who knows what it is supposed to be **(3-4)**	The code is poorly organized and very difficult to read. **(1-2)**
Documentation on (15%)	The team provides written documentation that clarifies their experimental results	Written documentation is generally complete, but occasional omissions create some lack of	Written documentation is incomplete, some lack of clarity.	There is no supporting written documentation.

	for that week. Objectives for the past week are clearly stated in relation to project goals, and the report is structured around their accomplishment. **(13-15)**	clarity. Objectives for the past week are included with no relationship to project goals. **(9-12)**	Objectives for the past week are included with no relationship to project goals. **(5-8)**	**(1-4)**
Total: 100%				